

Systemprogrammierung Prüfung, WS25/26

Mathias Lampert

31. März 2026

Inhaltsverzeichnis

1	C-Sprache	2
1.1	Bytespeicher für <code>uint8_t*</code> und <code>uint16_t*</code>	2
1.2	Einsatz von <code>static</code> in C	2
1.3	Endianness	2
2	Konfiguration	3
3	Bitmaskierung	4
3.1	Aufgabe: <code>setCtrl</code> -Funktion	4
3.2	Aufgabe: Rotate Right	4
4	Dynamische Speicherverwaltung	5
4.1	Aufgabe: <code>mem_calcStatistics</code>	5
5	Multitasking	6
5.1	Semaphor	6
5.2	Semaphor in FreeRTOS	6
5.3	Producer/Consumer Implementierung	7
5.4	Scheduling Diagramm	7
5.5	Producer/Consumer Erweiterung	8

1 C-Sprache

1.1 Bytespeicher für `uint8_t*` und `uint16_t*`

Beide Pointer belegen den gleichen Speicherplatz. Die Größe eines Pointers hängt von der Systemarchitektur ab. Ein 32-Bit-System verwendet 4 Bytes für eine Adresse. Ein 64-Bit-System verwendet 8 Bytes. Ein Pointer speichert ausschließlich die Speicheradresse. Der Datentyp des Ziels definiert die Interpretation der Daten. Der Datentyp ändert die Größe des Pointers nicht.

1.2 Einsatz von `static` in C

- Lokale Variablen: Die Variable behält ihren Wert über das Funktionsende hinaus. Das System speichert sie im Datensegment statt auf dem Stack. Beispiel: `static int counter = 0;` in einer Funktion.
- Globale Variablen: Das System beschränkt den Zugriff auf die aktuelle Quelldatei. Andere Dateien sehen diese Variable nicht. Beispiel: `static int file_internal_status;`
- Funktionen: Die Funktion ist nur innerhalb der eigenen C-Datei aufrufbar. Das verhindert Namenskonflikte beim Linken. Beispiel: `static void helper(void);`

1.3 Endianness

Endianness definiert die Byte-Reihenfolge im Speicher. Little Endian speichert das niederwertigste Byte an der kleinsten Adresse. RISC-V nutzt dieses Format. Big Endian speichert das höchstwertigste Byte zuerst. Portierungsprobleme treten auf; wenn Software Daten über Netzwerke schickt oder binäre Dateiformate zwischen Systemen austauscht. Sie müssen die Byte-Reihenfolge in diesen Fällen manuell umkehren.

2 Konfiguration

```
#include <stdio.h>
#include <stdint.h>

typedef struct {
    uint8_t mode;
    uint8_t level;
    uint8_t threshold;
    uint8_t flags;
} SensorConfig;

typedef union {
    uint32_t raw;
    SensorConfig config;
} SensorData;

void printSensorData(SensorData *data) {
    printf("Raw-Wert: 0x%08X\n", data->raw);
    printf("Mode: %u, Level: %u, Threshold: %u, Flags: %u\n",
           data->config.mode,
           data->config.level,
           data->config.threshold,
           data->config.flags);
}
```

3 Bitmaskierung

3.1 Aufgabe: setCtrl-Funktion

```
void setCtrl(uint32_t* pCtrl, bool enableMotor, bool enableLight, bool enableSensors) {
    uint32_t mask = (1 << 0) | (1 << 8) | (1 << 16);
    *pCtrl &= ~mask;
    if (enableMotor)    *pCtrl |= (1 << 0);
    if (enableLight)    *pCtrl |= (1 << 8);
    if (enableSensors) *pCtrl |= (1 << 16);
}
```

3.2 Aufgabe: Rotate Right

```
uint16_t rotateRight(uint16_t value, uint8_t n) {
    n %= 16;
    return (value >> n) | (value << (16 - n));
}
```

4 Dynamische Speicherverwaltung

4.1 Aufgabe: mem_calcStatistics

```
void mem_calcStatistics(float* avgAlloc, float* avgFree) {
    uint32_t sumAlloc = 0, countAlloc = 0;
    uint32_t sumFree = 0, countFree = 0;
    uint32_t offset = 0;

    while (offset < HEAPSIZ) {
        struct Header* h = (struct Header*)&gHeap[offset];
        if (h->flags & 0x01) {
            sumAlloc += h->length;
            countAlloc++;
        } else {
            sumFree += h->length;
            countFree++;
        }
        offset += sizeof(struct Header) + h->length;
    }
    *avgAlloc = countAlloc ? (float)sumAlloc / countAlloc : 0;
    *avgFree = countFree ? (float)sumFree / countFree : 0;
}
```

5 Multitasking

5.1 Semaphor

Ein Semaphor steuert den Zugriff auf Ressourcen. Er nutzt einen Zähler.

- P() (Take): Verringert den Zähler. Ist der Zähler Null; wartet der Task.
- V() (Give): Erhöht den Zähler. Das System weckt einen wartenden Task auf.

5.2 Semaphor in FreeRTOS

- xSemaphoreCreateCounting: Erstellt einen zählenden Semaphor. uxMaxCount legt den maximalen Zählerstand fest. uxInitialCount definiert den Startwert.
- xSemaphoreGive: Gibt den Semaphor frei; erhöht den Zähler. Rückgabe ist pdTRUE bei Erfolg.
- xSemaphoreTake: Reserviert den Semaphor; verringert den Zähler. xTicksToWait definiert die maximale Wartezeit.

5.3 Producer/Consumer Implementierung

```
SemaphoreHandle_t xSem;

void producerTaskFunc(void * pvParameters) {
    for (;;) {
        xSemaphoreGive(xSem);
        vTaskDelay(pdMS_TO_TICKS(30));
        xSemaphoreGive(xSem);
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

void consumerTaskFunc(void * pvParameters) {
    for (;;) {
        if (xSemaphoreTake(xSem, portMAX_DELAY)) {
            printf("Event verarbeitet\n");
        }
    }
}

void setupAndStartTasks(void) {
    xSem = xSemaphoreCreateCounting(10, 0);
    xTaskCreate(producerTaskFunc, "Prod", 2048, NULL, 2, NULL);
    xTaskCreate(consumerTaskFunc, "Cons", 2048, NULL, 1, NULL);
}
```

5.4 Scheduling Diagramm

Ein Tick entspricht 10 ms. Der Idle-Task nutzt die Zeit; wenn Producer und Consumer auf ihre Delays oder Semaphore warten.

- 0-10ms: Producer läuft; Consumer läuft; beide blockieren. Idle übernimmt.
- 30ms: Producer wacht auf; gibt Semaphor; blockiert. Consumer wacht auf; arbeitet; blockiert.

5.5 Producer/Consumer Erweiterung

Sie können folgende Strukturen nutzen:

- Message Queues für den Datenaustausch.
- Stream Buffer für kontinuierliche Daten.
- Task Notifications für einfache Signale.